

UNIT – III
Chapter - 6
Error Detection & Correction

1. What is Error Correction and Detection?

- **Error Detection**

- Whenever a message is transmitted, it may get scrambled by noise or data may get corrupted.
- To avoid this, we use error-detecting codes which are additional data added to a given digital message to help us detect if any error has occurred during transmission of the message.

- **Error Correction :**

- Send additional information so incorrect data can be corrected and accepted.
- Error correction is the additional ability to reconstruct the original, error-free data.

2. What are Transmission Errors?

- In Communication Networks External electromagnetic signals can cause incorrect delivery of data. By this, data in the communication Networks can be received incorrectly, data can be lost or unwanted Communication Networks data can be generated. Any of these problems are called transmission errors in communication networks.
- **Error**
- A condition when the receiver's information does not matches with the sender's information. During transmission, digital signals suffer from noise that can introduce errors in the binary bits travelling from sender to receiver. That means a 0 bit may change to 1 or a 1 bit may change to

3. Explain different types of error

- **Types of Errors**
- There may be three types of errors:
a) Single bit error



- In a frame, there is only one bit, anywhere though, which is corrupt.

b) **Multiple bits error**



Frame is received with more than one bits in corrupted state.

c) **Burst error**



Frame contains more than 1 consecutive bits corrupted.

4. Compare LRC and VRC.

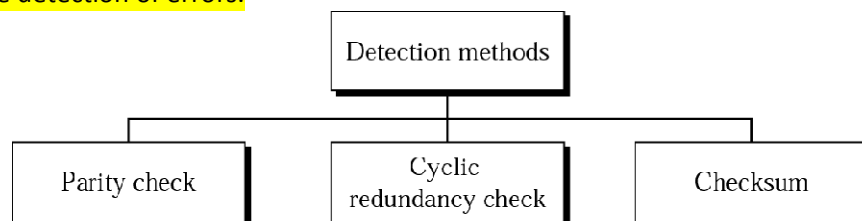
S.N	LRC	VRC
1	LRC can detect burst errors.	VRC is not capable of checking the burst error. It is capable of detecting Single bit error
2	LRC is also known as 2Dparity checker.	VRC is also known as odd parity checker
3	The advantage of using LRC over VRC is that it can check all the burst errors.	The advantage of using VRC is that it can checks all single bit errors but can check odd parity only in the case of change of odd bits.

5. Compare LRC and CRC

LRC	CRC
Longitudinal Redundancy Check (LRC) is a method in which a block of bits is organized in table(rows and columns)calculate the parity bit for each column and the set of this parity bit is also sending with original data. From the block of parity we can check the redundancy	Cyclic Redundancy Check (CRC) is one of the most common and powerful error detecting codes in which a sequence of redundant bits, called the CRC is appended to the end of the unit so that the resulting data unit become exactly divisible by a second, predetermined binary number.
LRC of n bits can easily detect	CRC is more powerful than
Burst error of n bits.	VRC and LRC in detecting errors.
A longitudinal redundancy check (LRC) is an error-detection method based on binary addition	CRC is based on binary division.

Error Detection

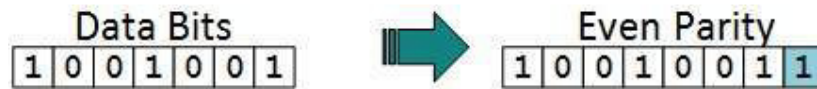
- Basic approach used for error detection is the use of redundancy bits, where additional bits are added to facilitate detection of errors.



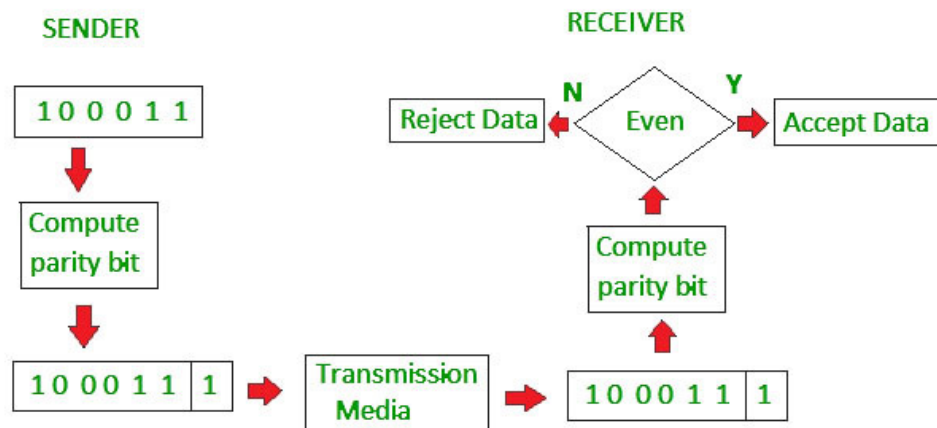
- Some popular techniques for error detection are:
 1. Simple Parity check
 2. Two-dimensional Parity check
 3. Checksum
 4. Cyclic redundancy check

6. Explain Parity Check error detection method

- One extra bit is sent along with the original bits to make number of 1s either even in case of even parity, or odd in case of odd parity.
- The sender while creating a frame counts the number of 1s in it. For example, if even parity is used and number of 1s is even then one bit with value 0 is added.
- This way number of 1s remains even. If the number of 1s is odd, to make it even a bit with value 1 is added.

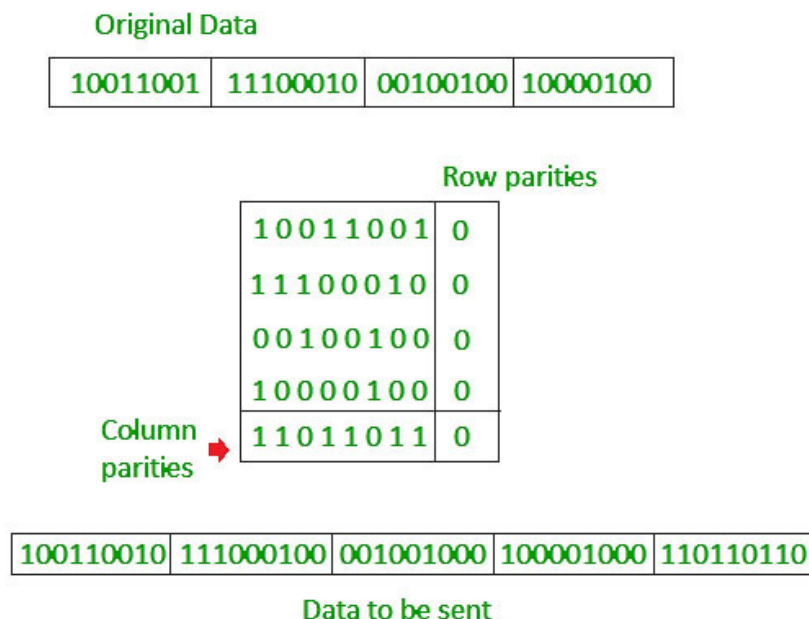


- The receiver simply counts the number of 1s in a frame. If the count of 1s is even and even parity is used, the frame is considered to be not-corrupted and is accepted.
- If the count of 1s is odd and odd parity is used, the frame is still not corrupted.
- If a single bit flips in transit, the receiver can detect it by counting the number of 1s. But when more than one bits are erroneous, then it is very hard for the receiver to detect the error.



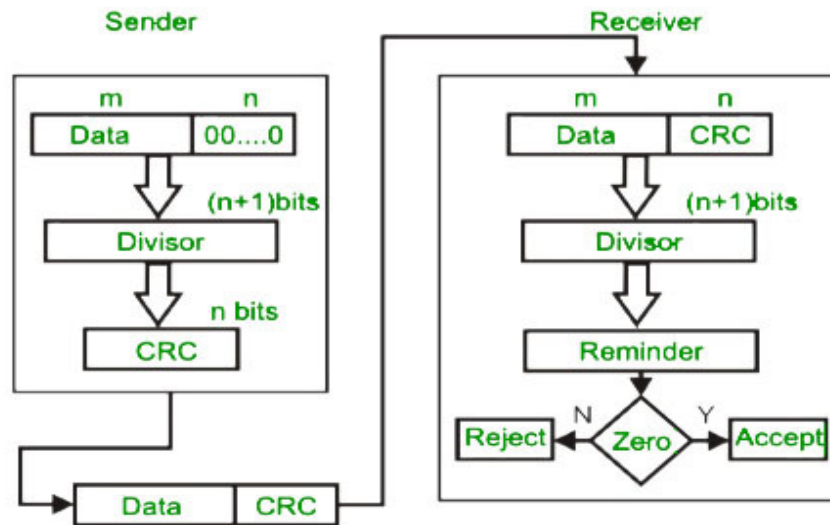
7. Explain Two-dimensional Parity check error detection method

- Parity check bits are calculated for each row, which is equivalent to a simple parity check bit.
- Parity check bits are also calculated for all columns, then both are sent along with the data.
- At the receiving end these are compared with the parity bits calculated on the received data.

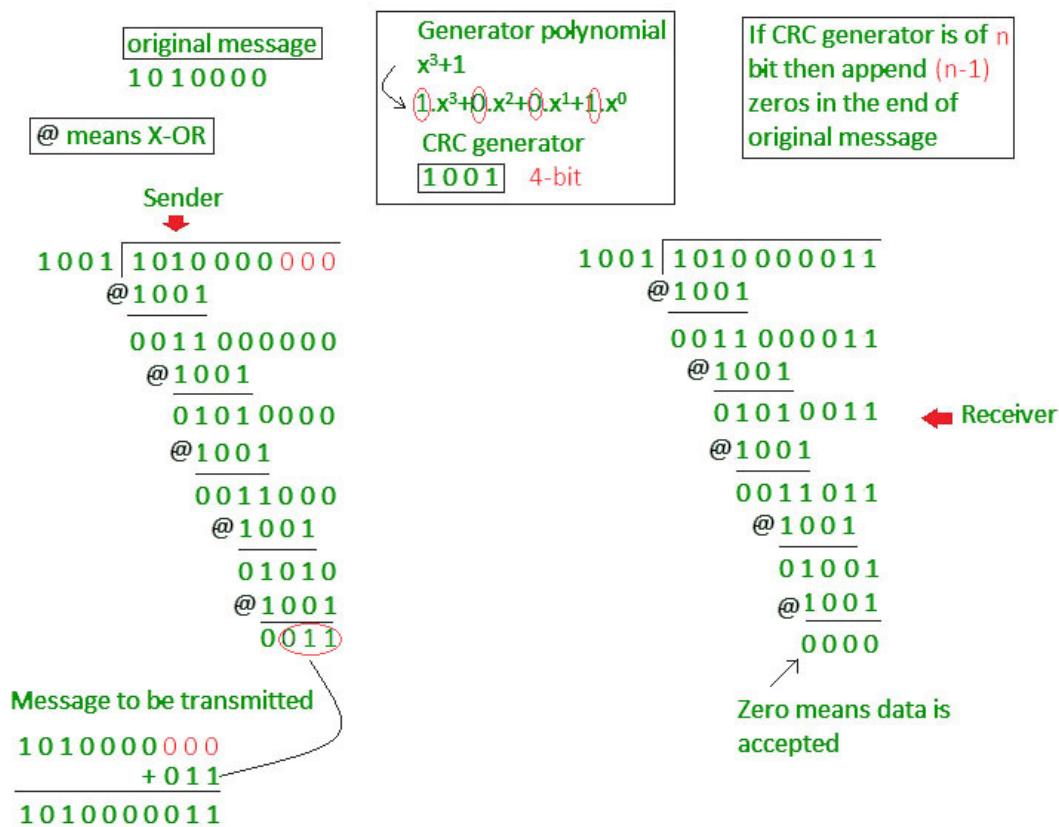


8. Explain Cyclic redundancy check (CRC) error detection method

- Unlike checksum scheme, which is based on addition, CRC is based on binary division.
- In CRC, a sequence of redundant bits, called cyclic redundancy check bits, are appended to the end of data unit so that the resulting data unit becomes exactly divisible by a second, predetermined binary number.
- At the destination, the incoming data unit is divided by the same number. If at this step there is no remainder, the data unit is assumed to be correct and is therefore accepted.
- A remainder indicates that the data unit has been damaged in transit and therefore must be rejected.



Example



Error Correction

- **Backward Error Correction**
- When the receiver detects an error in the data received, it requests back the sender to retransmit the data unit.
- **Forward Error Correction**
- When the receiver detects some error in the data received, it executes error-correcting code, which helps it to auto-recover and to correct some kinds of errors.

9. Hamming Code

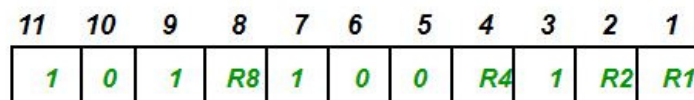
- **General Algorithm of Hamming code**
- The Hamming Code is simply the use of extra parity bits to allow the identification of an error.
- Write the bit positions starting from 1 in binary form (1, 10, 11, 100, etc).i.e. 2^n
- All the bit positions that are a power of 2 are marked as parity bits (1, 2, 4, 8, etc).
- All the other bit positions are marked as data bits.
- Each data bit is included in a unique set of parity bits, as determined its bit position in binary form.
 - a. Parity bit 1 covers all the bits positions whose binary representation includes a 1 in the least significant position (1, 3, 5, 7, 9, 11, etc).
 - b. Parity bit 2 covers all the bits positions whose binary representation includes a 1 in the second position from the least significant bit (2, 3, 6, 7, 10, 11, etc).
 - c. Parity bit 4 covers all the bits positions whose binary representation includes a 1 in the third position from the least significant bit (4–7, 12–15, 20–23, etc).
 - d. Parity bit 8 covers all the bits positions whose binary representation includes a 1 in the fourth position from the least significant bit (8–15, 24–31, 40–47, etc).
 - e. In general each parity bit covers all bits where the bitwise AND of the parity position and the bit position is non-zero.

Since we check for even parity set a parity bit to 1 if the total number of ones in the positions it checks is odd. Set a parity bit to 0 if the total number of ones in the positions it checks is even.

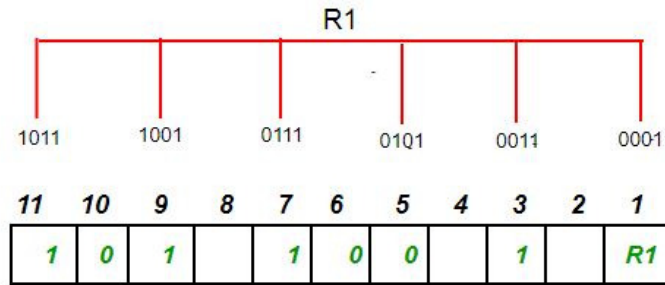
- **Determining the position of redundant bits –**
- These redundancy bits are placed at the positions which correspond to the power of 2. As in the above example:
- The number of data bits = 7
- The number of redundant bits = 4
- The total number of bits = 11
- The redundant bits are placed at positions corresponding to power of 2- 1, 2, 4, and 8



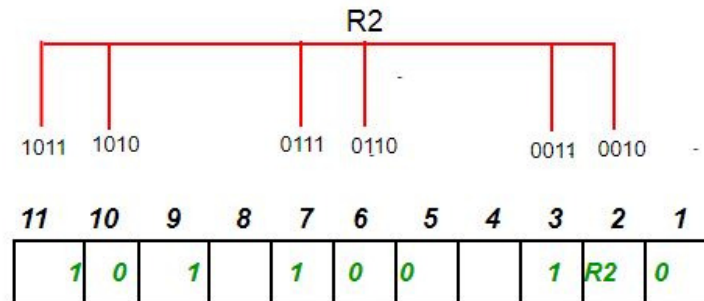
- Suppose the data to be transmitted is 1011001, the bits will be placed as follows:



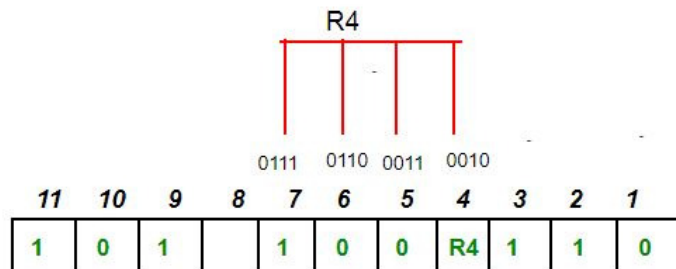
- **Determining the Parity bits –**
- R1 bit is calculated using parity check at all the bits positions whose binary representation includes a 1 in the least significant position. R1: bits 1, 3, 5, 7, 9, 11



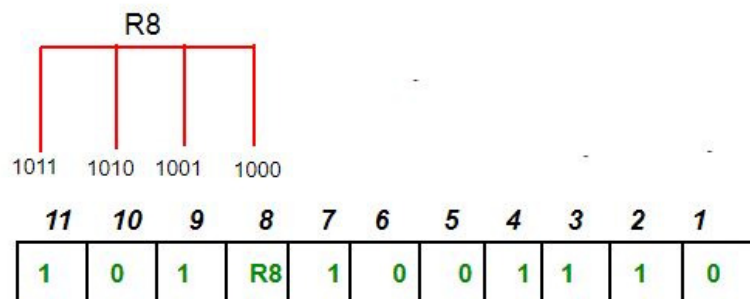
- R2 bit is calculated using parity check at all the bits positions whose binary representation includes a 1 in the second position from the least significant bit. R2: bits 2,3,6,7,10,11
- To find the redundant bit R2, we check for even parity. Since the total number of 1's in all the bit positions corresponding to R2 is an odd number the value of R2 (parity bit's value)=1



- To find the redundant bit R1, we check for even parity. Since the total number of 1's in all the bit positions corresponding to R1 is an even number the value of R1 (parity bit's value) = 0
- R4 bit is calculated using parity check at all the bits positions whose binary representation includes a 1 in the third position from the least significant bit. R4: bits 4, 5, 6, 7



- To find the redundant bit R4, we check for even parity. Since the total number of 1's in all the bit positions corresponding to R4 is an odd number the value of R4 (parity bit's value) = 1
- R8 bit is calculated using parity check at all the bits positions whose binary representation includes a 1 in the fourth position from the least significant bit. R8: bit 8,9,10,11



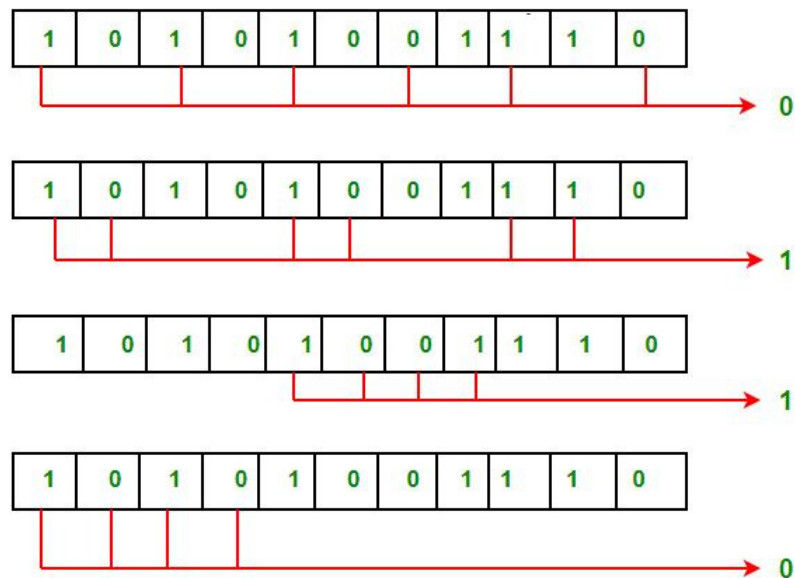
- To find the redundant bit R8, we check for even parity. Since the total number of 1's in all the bit positions corresponding to R8 is an even number the value of R8 (parity bit's value)=0.
- To find the redundant bit R8, we check for even parity. Since the total number of 1's in all the bit positions corresponding to R8 is an even number the value of R8 (parity bit's value)=0.
- Thus, the data transferred is:

11	10	9	8	7	6	5	4	3	2	1
1	0	1	0	1	0	0	1	1	1	0

- To find the redundant bit R8, we check for even parity. Since the total number of 1's in all the bit positions corresponding to R8 is an even number the value of R8 (parity bit's value)=0.
- Thus, the data transferred is:

• **Error detection and correction –**

- Suppose in the above example the 6th bit is changed from 0 to 1 during data transmission, then it gives new parity values in the binary number:



- The bits give the binary number as 0110 whose decimal representation is 6. Thus, the bit 6 contains an error. To correct the error the 6th bit is changed from 1 to 0.